Plan de desarrollo — Práctica 002

SVM de regresión (SVR) para predecir ventas diarias

Campo	Valor
Asignatura	Aprendizaje Automático
Práctica	SVM y explicación de predicciones — alcance exclusivo: solo SVM/SVR
Equipo	4 integrantes
Duración estimada	2 semanas, 5 sesiones de trabajo + 1 sesión de buffer
Repositorio	`tareas/practica-002`
Dataset	`tareas/practica-001/datos/ventas_features.csv`

1. Resumen ejecutivo

El objetivo de esta práctica es desarrollar, optimizar, evaluar y explicar un modelo de **Máquinas de Soporte Vectorial (SVM)**. La guía permite explícitamente problemas de **regresión y/o clasificación**. Como la variable de interés del dataset es `cantidad_vendida` (continua, en unidades por día), la solución técnicamente correcta es **SVR (Support Vector Regression)** y no una clasificación artificial.

El equipo construirá un único notebook reproducible que:

1. Carga y prepara el dataset de la práctica 001.
2. Entrena SVR con distintos **kernels** (`linear` , `rbf` , `poly` , `sigmoid`) y distintos valores de **penalización** `C` .
3. Registra **todos** los entrenamientos en **MLflow** y consolida los resultados en una lista/tabla comparativa.
4. Optimiza el modelo con `GridSearchCV` (C, epsilon, gamma, degree).
5. Define una función `predecir_venta()` para un sample nuevo, probada con **dos casos** reales del dataset (demanda alta y demanda baja).
6. Justifica las predicciones con **permutation importance**, **SHAP** y **PDP/ICE**.
7. Cierra con conclusiones basadas en métricas y referencias en **APA 7**.

Resultado esperado (medido previamente sobre este dataset):

Modelo	RMSE	MAE	R^2
Baseline ingenuo (`promedio_movil_7d`)	41.72	31.42	—
SVR linear, C=1, split temporal	26.27	20.12	0.768
SVR RBF, C=10, split aleatorio	22.42	16.68	0.848
LSTM de la práctica 001 (referencia)	23.19	17.30	—

La conclusión esperada es que un SVR simple alcanza un desempeño comparable al LSTM de la práctica anterior con una fracción de la complejidad.

---

2. Objetivo, alcance y exclusiones

2.1 Objetivo general

Aplicar SVR al dataset de ventas retail, optimizar sus hiperparámetros, evaluar su desempeño, exponer una función de predicción reutilizable y justificar las predicciones con técnicas de interpretabilidad.

2.2 Alcance (lo que sí se entrega)

Ítem de la rúbrica	Puntos	Evidencia en el notebook
3.1 Desarrollo y optimización de modelos	2	Barrido de kernels × C, `GridSearchCV`, tabla y figuras comparativas
3.2 Justificación de predicciones	2	Permutation importance, SHAP global y local, PDP/ICE
3.3 Función de predicción con nuevo sample	2	`predecir_venta()` probada con dos casos distintos
3.4 Conclusiones y referencias	2	Conclusiones con métricas + referencias APA 7

> **Alcance exclusivo:** esta práctica se limita a **SVM/SVR y su explicabilidad**. No se desarrolla **nada** de **Random Forest** ni **árboles de decisión**, incluido el ejercicio de Information Gain. Ese contenido queda fuera del alcance y no se planifica ni se incluye como comparación.

2.3 Fuera de alcance

- **Random Forest** y **árboles de decisión** (incluido Information Gain): no se implementan, no se comparan y no forman parte de ningún entregable.
- Despliegue web o API (la práctica 001 ya cubrió eso).
- Entrenamiento en GPU o deep learning.
- Búsqueda exhaustiva de hiperparámetros tipo bayesiano.
- Reentrenamiento con datos nuevos o automatización MLOps.

---

3. Decisiones técnicas cerradas

Estas decisiones se toman una vez y no se reabren sin acuerdo del equipo.

Decisión	Valor	Justificación
Variable objetivo	`cantidad_vendida`	Es la variable de interés del dataset y evita clases artificiales
Tipo de modelo	`SVR` de scikit-learn	SVM para regresión, permitido por la guía

Decisión	Valor	Justificación
Split	Temporal 80/20 por fecha	Evita fuga entre días consecutivos; simula predecir el futuro
Escalado de X	`StandardScaler`	SVR es sensible a la escala
Escalado de y	`TransformedTargetRegressor` + `StandardScaler`	`epsilon` y `C` dependen de la escala del target
Kernels	`linear` , `rbf` , `poly` , `sigmoid`	Requisito explícito de la guía
Penalización C	`0.1, 1, 10, 100`	Requisito explícito; rango suficiente para ver sobre-ajuste
Búsqueda fina	`GridSearchCV` 5-fold sobre C, epsilon, gamma, degree	Optimización de la sección 3.1
Métrica principal	RMSE	Misma unidad que la variable (unidades vendidas)
Métricas secunda-rias	MAE, R^2, MAPE	Complementan la interpretación
Semilla	`random_state=42`	Reproducibilidad
Tracking	MLflow local (`./mlruns`)	Requisito de la guía
Explicabilidad	Permutation importance, SHAP, PDP/ICE	Cubre visualización y algoritmos

3.1 Variables de entrada

Numéricas (11): `mes` , `dia_del_mes` , `dia_semana` , `es_fin_de_semana` , `es_feriado` , `sema-na_del_periodo` , `ventas_dia_anterior` , `ventas_hace_7_dias` , `promedio_movil_7d` , `stock_en_momento` , `dias_para_expirar` .

Categóricas (2, one-hot): `categoria` , `product_id` .

Excluidas y por qué:

Columna	Motivo
`stock_actual`	Constante por producto; identifica al producto, no aporta señal
`nombre` , `unidad`	Texto redundante con `product_id` y `categoria`
`fecha_venta` , `anio` , `nombre_dia` , `tipo_dia`	Redundantes con las variables de calendario
`ventas_hace_7_dias` (nulos)	Se eliminan las filas con nulos (600 → ~530)

> Regla de oro: ninguna variable puede contener información del futuro ni del target. `ven-tas_dia_anterior` y `promedio_movil_7d` son válidas porque solo usan días previos.

3.2 Advertencia sobre `categoria` como target

Se evaluó usar `categoria` como variable objetivo. No se recomienda:

- `Alimentos` concentra 300 filas (5 productos); las otras 5 categorías tienen **1 solo producto** cada una (60 filas).
- Sin variables de fuga, el F1-macro es ≈ 0.30.

- Con `stock_actual` o `dias_para_expirar`, el F1-macro sube a ≈ 0.97, pero es **fuga**: esas columnas son la huella del producto. El modelo identifica productos, no categorías.

Por eso `categoria` se usa como **variable predictora**, no como target.

---

4. Arquitectura del notebook

Archivo único: `svr_ventas.ipynb`. Regla: cada sección empieza con una celda Markdown que explica qué se hace y por qué.

#	Sección	Contenido	Rúbrica	Responsable
0	Portada	Título, integrantes, objetivo, dataset	—	P4
1	Configuración	Imports, rutas, semilla, versión de librerías	—	P1
2	Datos y EDA mínimo	Carga, nulos, distribución del target, ventas por categoría	—	P1
3	Features y split	Selección, one-hot, `dropna`, split temporal	—	P1
4	Pipeline SVR	`TransformedTargetRegressor`, `StandardScaler`, `SVR`	3.1	P2
5	Barrido kernels × C	16 runs en MLflow + métricas	3.1	P2
6	Optimización	`GridSearchCV`, mejor modelo, reentrenamiento	3.1	P2
7	Evaluación	RMSE/MAE/R^2/MAPE, predicho vs real, residuos	3.1	P3
8	Resultados consolidados	`mlflow.search_runs` → lista/tabla + CSV + figura	3.1	P3
9	Función de predicción	`predecir_venta()` + dos casos	3.3	P3
10	Explicabilidad	Permutation, SHAP global/local, PDP/ICE	3.2	P4
11	Conclusiones	Hallazgos, comparación con LSTM, limitaciones	3.4	P4
12	Referencias	APA 7	3.4	P4

---

5. Equipo, roles y responsabilidades

5.1 Perfiles

Rol	Integrante	Responsabilidad principal	Carga estimada
P1 — Datos y EDA	*(nombre)*	Dataset, features, split, control de fuga	8 h
P2 — Modelado y MLflow	*(nombre)*	Pipeline SVR, barrido, GridSearch, tracking	10 h
P3 — Evaluación y predicción	*(nombre)*	Métricas, figuras, `predecir_venta`, HTML	9 h
P4 — Explicabilidad y documentación	*(nombre)*	SHAP/PDP, conclusiones, APA, revisión final	10 h

5.2 Responsabilidades detalladas

Cada rol tiene un objetivo, tareas verificables, entregables, criterio de aceptación y un límite claro de lo que **no** le corresponde. Las horas son estimadas y suman 37 h de trabajo efectivo para el equipo.

5.2.1 P1 — Datos y EDA (8 h)

Objetivo: entregar un dataset limpio, un conjunto de variables sin fuga de información y un split reproducible que el resto del equipo pueda usar sin modificarlo.

Tareas concretas:

1. Crear la estructura `datos/`, `resultados/figuras/` y copiar `ventas_features.csv` desde la práctica 001.
2. Cargar el CSV con `parse_dates`, ordenar por fecha y producto, y validar dimensiones (600 filas, 10 productos × 60 días), tipos y rangos.
3. Hacer el EDA mínimo: distribución de `cantidad_vendida`, ventas por categoría y producto, evolución temporal y mapa de nulos.
4. Documentar los nulos de `ventas_dia_anterior` (10), `ventas_hace_7_dias` (70) y `promedio_movil_7d` (10), y justificar la decisión de eliminar esas filas (~530 registros útiles).
5. Definir las listas `NUM`, `CAT` y `TARGET`, y justificar cada exclusión (`stock_actual`, textos, columnas redundantes).
6. Construir `X` con one-hot encoding y `y` con `cantidad_vendida`; verificar que no queden nulos ni columnas constantes.
7. Congelar el split temporal 80/20 por fecha y guardar la lista `COLUMNAS`.
8. Ejecutar una revisión de fuga de información: comprobar que ningún predictor usa datos del futuro ni del target.
9. Escribir en Markdown las secciones 0–3 del notebook y guardar las figuras del EDA.

Entregables:

- Notebook secciones 0–3 ejecutando de inicio a fin.
- `resultados/figuras/eda_distribucion.png`, `eda_categoria.png`, `eda_temporal.png`.
- Lista `COLUMNAS` y tabla de variables incluidas/excluidas.
- Documento breve de decisiones de limpieza (puede ser una celda Markdown).

Criterio de aceptación:

- `X_train`, `X_test`, `y_train`, `y_test` existen, no tienen nulos y conservan la proporción temporal esperada.
- Cada exclusión de columna está justificada por escrito.
- P2 revisa y aprueba la ausencia de fuga antes de modelar.

Interacción: entrega a P2 la matriz y el split; P3 reutiliza las figuras del EDA. **No le corresponde:** entrenar modelos, calcular métricas, SHAP ni conclusiones.

Actividad	Horas
Estructura y copia del dataset	1.0
Carga, validación y EDA	2.0
Features, one-hot y target	2.0
Split temporal y revisión de fuga	1.5
Documentación y figuras	1.5
Total	**8.0**

5.2.2 P2 — Modelado y MLflow (10 h)

Objetivo: construir el pipeline SVR, ejecutar el barrido de kernels y penalización, optimizar los hiperparámetros y dejar todo registrado en MLflow de forma reproducible.

Tareas concretas:

1. Configurar MLflow con `tracking_uri="file:./mlruns"` y experimento `svr_ventas`; validar con un run de prueba.
2. Implementar `construir_svr(kernel, C, epsilon, gamma, degree)` usando `TransformedTargetRegressor` + `StandardScaler`.
3. Entrenar el baseline (`linear`, C=1) y documentar sus métricas como punto de partida.
4. Ejecutar el barrido de 4 kernels × 4 valores de C (16 runs), registrando parámetros, RMSE, MAE, R^2, MAPE y el modelo.
5. Detectar y documentar el comportamiento inestable de `sigmoid` sin ocultar resultados.
6. Ejecutar `GridSearchCV` 5-fold sobre C, epsilon, gamma y degree; registrar la búsqueda como un run adicional.
7. Seleccionar el mejor modelo y justificar la decisión comparando RMSE de CV contra RMSE de test.
8. Generar la tabla consolidada con `mlflow.search_runs` (o dejar la función lista para que P3 la exporte).
9. Documentar hiperparámetros, semilla y versión de scikit-learn.

Entregables:

- Notebook secciones 4–6 ejecutando.
- ≥ 17 runs visibles en `mlflow ui` (16 del barrido + 1 de optimización).
- Modelo final en memoria y artefacto en MLflow.
- Tabla de resultados cruda y mejores hiperparámetros.

Criterio de aceptación:

- Todos los runs tienen kernel, C y métricas completas.
- El mejor modelo se puede reconstruir con los parámetros registrados.
- La comparación CV vs test está explicada, no solo mostrada.

Interacción: depende del split de P1; entrega el modelo final a P3 y P4. **No le corresponde:** figuras de evaluación final, función de predicción, SHAP, conclusiones ni referencias.

Actividad	Horas
Configuración de MLflow	1.0
Pipeline y baseline	2.0
Barrido kernels × C	3.0
GridSearchCV y selección	2.5
Documentación y tabla	1.5
Total	**10.0**

5.2.3 P3 — Evaluación y predicción (9 h)

Objetivo: medir correctamente el modelo final, comunicar los resultados con figuras claras y entregar una función de predicción usable con dos casos demostrativos.

Tareas concretas:

1. Implementar las métricas RMSE, MAE, R^2 y MAPE en una función reutilizable.
2. Evaluar el mejor modelo sobre `X_test` y construir la tabla de métricas finales.
3. Generar las figuras: predicho vs real con recta `y = x`, histograma de residuos y barras comparativas de RMSE por kernel y por C.
4. Exportar la tabla consolidada de MLflow a `resultados/resultados_modelos.csv` y ordenarla por RMSE.
5. Implementar `predecir_venta(nuevo_sample)` alineando columnas con `COLUMNAS`, rellenando categóricas faltantes con 0 y devolviendo la predicción redondeada.
6. Diseñar dos casos con productos y contextos distintos: uno de demanda alta (fin de semana/feriado, lags altos) y uno de demanda baja (entre semana, lags bajos).
7. Comparar cada predicción con un valor real cercano del dataset y explicar la diferencia.
8. Probar casos límite (campo faltante, categoría no vista) y documentar el comportamiento.
9. Exportar el notebook a HTML y revisar que tablas y figuras se vean bien.

Entregables:

- Notebook secciones 7–9 ejecutando.
- `resultados/resultados_modelos.csv`.
- Figuras `predicho_vs_real.png`, `residuos.png`, `comparacion_kernels.png`.
- Función `predecir_venta()` con dos salidas documentadas.
- `svr_ventas.html` generado.

Criterio de aceptación:

- Las métricas del mejor modelo coinciden con las registradas en MLflow.
- La función responde para los dos casos y no falla con campos opcionales ausentes.
- El HTML se abre y muestra todas las secciones sin errores.

Interacción: recibe el modelo de P2; entrega los dos casos a P4 para los waterfall. **No le corresponde:** elegir hiperparámetros, interpretar SHAP ni redactar conclusiones.

Actividad	Horas
Métricas y tabla final	1.5
Figuras de evaluación	2.0
Consolidado desde MLflow	1.0
Función de predicción	2.0
Dos casos y pruebas límite	1.0
Exportación HTML y revisión	1.5
Total	**9.0**

5.2.4 P4 — Explicabilidad y documentación (10 h)

Objetivo: justificar las predicciones con evidencia visual y algorítmica, cerrar el trabajo con conclusiones basadas en métricas y dejar las referencias en APA 7.

Tareas concretas:

1. Preparar una explicación breve de qué es SHAP, qué es PDP/ICE y qué mide la importancia por permutación.
2. Calcular la importance por permutación con `n_repeats=20` y graficar el top-10.
3. Calcular SHAP global con 100 muestras de fondo y `nsamples=100`; guardar el summary plot.
4. Calcular SHAP local (waterfall) para los dos casos que definió P3 y guardar una figura por caso.
5. Calcular PDP/ICE para las dos variables más influyentes y guardar las figuras.
6. Interpretar cada figura en Markdown: qué variable empuja la predicción hacia arriba o hacia abajo y si tiene sentido para el negocio.
7. Redactar las conclusiones con cifras: mejor kernel/C, comparación con el baseline y con el LSTM de la práctica 001, y limitaciones del dataset.
8. Construir la lista de referencias en APA 7 y citarlas en el texto.
9. Hacer la revisión final contra la rúbrica y coordinar con P3 la exportación HTML.

Entregables:

- Notebook secciones 10–12 ejecutando.
- `resultados/figuras/permutation_importance.png`, `shap_summary.png`, `shap_caso_alto.png`, `shap_caso_bajo.png`, `pdp_ice.png`.
- Conclusiones con métricas y referencias APA 7.
- Checklist de rúbrica marcado.

Criterio de aceptación:

- Cada figura tiene una interpretación escrita, no solo el gráfico.
- Las conclusiones citan números concretos (RMSE, R^2, kernel, C).
- Las referencias están en APA 7 y aparecen citadas en el texto.
- La revisión final no deja ítems de la rúbrica sin evidencia.

Interacción: depende del modelo de P2 y de los dos casos de P3. **No le corresponde:** modificar el pipeline, recalcular métricas base ni cambiar el split.

Actividad	Horas
Fundamentos y permutation importance	2.0
SHAP global y local	3.0
PDP/ICE e interpretación	1.5
Conclusiones	1.5
Referencias APA y revisión final	2.0
Total	**10.0**

5.3 Matriz RACI

`R` = responsable de ejecutar, `A` = aprueba, `C` = consultado, `I` = informado.

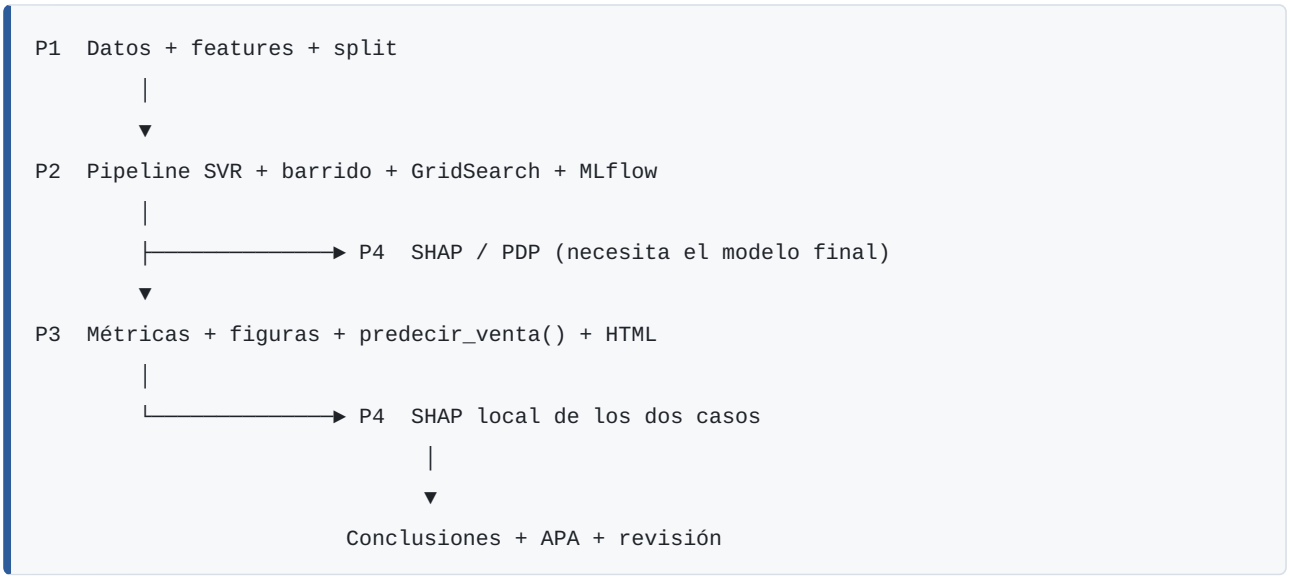
Tarea	P1	P2	P3	P4
Estructura del repo y entorno	**R**	C	I	I
Carga, EDA y features	**R**	C	I	I
Split temporal y control de fuga	**R**	A	C	C
Pipeline SVR y escalado del target	C	**R**	C	I
Barrido kernels × C	I	**R**	C	I
GridSearchCV	I	**R**	C	I
Métricas y figuras de evaluación	I	C	**R**	C
Tabla consolidada desde MLflow	I	**R**	C	I
`predecir_venta()` y dos casos	I	C	**R**	C
Permutation importance	I	C	C	**R**
SHAP y PDP/ICE	I	C	C	**R**
Conclusiones	C	C	C	**R**
Referencias APA	I	I	I	**R**
Exportación HTML y revisión de rúbrica	I	I	**R**	A

5.4 Regla de edición del notebook

El notebook es un archivo único y cuatro personas no pueden editarlo a la vez.

1. Cada integrante tiene su rama: `feat/datos`, `feat/modelos`, `feat/evaluacion`, `feat/explicabilidad`.
2. Solo se edita la sección asignada.
3. Antes de editar: `git pull --rebase origin main`.
4. Al terminar: commit pequeño y push; el integrador (P1) hace el merge.
5. Si hay conflicto, se resuelve en pareja (autor + integrador) y se vuelve a ejecutar el notebook completo.
6. Prohibido dejar celdas con errores o salidas obsoletas en `main`.

5.5 Flujo de dependencias entre roles



Reglas del flujo:

- P1 no entrega hasta que P2 confirme que no hay fuga de información.
- P2 no entrega hasta que la tabla de MLflow esté completa.
- P3 no entrega hasta que la función funcione con los dos casos.
- P4 no entrega hasta que cada figura tenga interpretación y la rúbrica esté marcada.
- Ningún rol modifica el trabajo de otro sin avisar y sin volver a ejecutar el notebook.

---

6. Cronograma

6.1 Sesiones

Sesión	Duración	Objetivo	Actividades	Responsable	Entregable
S1	2 h	Base reproducible	Kickoff, repo, entorno, carga, EDA, features, split	P1	Notebook secciones 0–3 ejecutando
S2	2 h	Primer modelo	Pipeline SVR, baseline, configuración de MLflow, 4 runs	P2	Sección 4–5 con 4 runs
S3	2 h	Optimización	16 runs, `GridSearchCV`, tabla consolidada, figuras	P2 + P3	Secciones 5–8 completas
S4	2 h	Predicción	Métricas finales, `predecir_venta()`, dos casos, permutation importance	P3 + P4	Secciones 7, 9 y 10.1
S5	2 h	Explicabilidad y cierre	SHAP, PDP/ICE, conclusiones, APA, exportación HTML	P4 + todos	Notebook completo + HTML
S6	1 h	Buffer	Correcciones, ensayo de sustentación	Todos	Versión final

6.2 Hitos

Hito	Fecha	Criterio de cumplimiento
M1	Fin S1	El notebook carga datos y genera `X_train`, `X_test` sin errores
M2	Fin S3	≥ 16 runs visibles en `mlflow ui` y tabla `resultados_modelos.csv`
M3	Fin S4	`predecir_venta()` devuelve valores para dos casos distintos
M4	Fin S5	Notebook ejecutado de inicio a fin, HTML generado, rúbrica completa

6.3 Trabajo entre sesiones

Entre	Tarea	Responsable	Tiempo
S1–S2	Revisar dokumentación de SVR y preparar el pipeline	P2	2 h
S2–S3	Preparar funciones de métricas y figuras	P3	2 h
S3–S4	Investigar SHAP con SVR y preparar el guion de conclusiones	P4	3 h
S4–S5	Redactar borrador de conclusiones y referencias	P4	2 h
S5–S6	Ensayo individual de la sustentación	Todos	1 h

---

7. Guía técnica paso a paso

7.1 Preparar el entorno

Desde `practica-002`:

```
python3.12 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install pandas numpy scikit-learn mlflow shap matplotlib seaborn jupyter nbconvert
.venv/bin/python -m pip freeze > requirements.txt
```

Verificación rápida:

```
.venv/bin/python -c "import sklearn, mlflow, shap; print(sklearn.__version__, mlflow.__version__, shap.__version__)"
```

7.2 Estructura de carpetas

```
practica-002/
├── datos/
│   └── ventas_features.csv        # copia del dataset de la práctica 001
├── resultados/
│   ├── resultados_modelos.csv
│   └── figuras/
├── mlruns/                        # se crea solo al registrar en MLflow
├── svr_ventas.ipynb
├── svr_ventas.html
├── requirements.txt
└── README.md
```

Copiar el dataset:

```
mkdir -p datos resultados/figuras
cp ../practica-001/datos/ventas_features.csv datos/
```

7.3 Carga, features y target

```
import pandas as pd

df = pd.read_csv("datos/ventas_features.csv", parse_dates=["fecha_venta"])
df = df.sort_values(["fecha_venta", "product_id"]).reset_index(drop=True)

NUM = [
    "mes", "dia_del_mes", "dia_semana", "es_fin_de_semana", "es_feriado",
    "semana_del_periodo", "ventas_dia_anterior", "ventas_hace_7_dias",
    "promedio_movil_7d", "stock_en_momento", "dias_para_expirar",
]
CAT = ["categoria", "product_id"]
TARGET = "cantidad_vendida"

df = df.dropna(subset=NUM + [TARGET]).reset_index(drop=True)

X = pd.get_dummies(df[NUM + CAT], columns=CAT, drop_first=True, dtype=float)
y = df[TARGET]

print(df.shape, X.shape)
```

7.4 Split temporal

```
corte = df["fecha_venta"].quantile(0.8)
train = df["fecha_venta"] <= corte
test = ~train

X_train, y_train = X[train], y[train]
X_test, y_test = X[test], y[test]

print("Corte:", corte.date())
print("Train:", train.sum(), "Test:", test.sum())
```

Guardar `COLUMNAS = X_train.columns.tolist()` porque la función de predicción lo necesita.

7.5 Pipeline SVR con target escalado

```
from sklearn.compose import TransformedTargetRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

def construir_svr(kernel="rbf", C=1.0, epsilon=0.1, gamma="scale", degree=3):
    return TransformedTargetRegressor(
        regressor=make_pipeline(
            StandardScaler(),
            SVR(kernel=kernel, C=C, epsilon=epsilon, gamma=gamma, degree=degree),
        ),
        transformer=StandardScaler(),
    )
```

> ¿Por qué escalar `y`? `epsilon` define una banda de tolerancia en unidades del target. Con ventas de 67–411, `epsilon=0.1` obliga al modelo a ajustar casi exactamente cada punto y sobreajusta. Escalar el target hace que `epsilon` y `C` sean interpretables y comparables entre corridas.

7.6 Barrido kernels × C con MLflow

```
import numpy as np
import mlflow
import mlflow.sklearn
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    mean_absolute_percentage_error,
)

mlflow.set_tracking_uri("file:./mlruns")
mlflow.set_experiment("svr_ventas")

for kernel in ["linear", "rbf", "poly", "sigmoid"]:
    for C in [0.1, 1, 10, 100]:
        with mlflow.start_run(run_name=f"{kernel}_C{C}"):
            modelo = construir_svr(kernel=kernel, C=C)
            modelo.fit(X_train, y_train)
            pred = modelo.predict(X_test)

            rmse = float(np.sqrt(mean_squared_error(y_test, pred)))
            mae = float(mean_absolute_error(y_test, pred))
            r2 = float(r2_score(y_test, pred))
            mape = float(mean_absolute_percentage_error(y_test, pred) * 100)

            mlflow.log_params({
                "kernel": kernel, "C": C, "epsilon": 0.1, "gamma": "scale",
                "split": "temporal", "target": TARGET,
            })
            mlflow.log_metrics({"rmse": rmse, "mae": mae, "r2": r2, "mape": mape})
            mlflow.sklearn.log_model(modelo, "modelo")
```

> `sigmoid` puede producir valores extremos. Es un resultado esperado y sirve para la discusión: no todos los kernels son adecuados para datos tabulares.

7.7 Optimización con GridSearchCV

```
from sklearn.model_selection import GridSearchCV, KFold

param_grid = [
    {
        "regressor__svr__kernel": ["linear"],
        "regressor__svr__C": [0.1, 1, 10, 100],
        "regressor__svr__epsilon": [0.01, 0.1, 0.5],
    },
    {
        "regressor__svr__kernel": ["rbf"],
        "regressor__svr__C": [0.1, 1, 10, 100],
        "regressor__svr__epsilon": [0.01, 0.1, 0.5],
        "regressor__svr__gamma": ["scale", 0.01, 0.1],
    },
    {
        "regressor__svr__kernel": ["poly"],
        "regressor__svr__C": [0.1, 1, 10],
        "regressor__svr__epsilon": [0.1, 0.5],
        "regressor__svr__gamma": ["scale", 0.1],
        "regressor__svr__degree": [2, 3],
    },
]

gs = GridSearchCV(
    construir_svr(),
    param_grid,
    cv=KFold(5, shuffle=True, random_state=42),
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
)
gs.fit(X_train, y_train)

mejor_modelo = gs.best_estimator_
print("Mejores parámetros:", gs.best_params_)
print("RMSE CV:", -gs.best_score_)
```

Registrar también esta búsqueda como un run de MLflow (parámetros + métricas + modelo).

7.8 Consolidar resultados desde MLflow

```
runs = mlflow.search_runs(experiment_names=["svr_ventas"])

tabla = runs[[
    "tags.mlflow.runName", "params.kernel", "params.C",
    "metrics.rmse", "metrics.mae", "metrics.r2", "metrics.mape",
]].copy()
tabla.columns = ["run", "kernel", "C", "RMSE", "MAE", "R2", "MAPE"]
tabla = tabla.sort_values("RMSE").reset_index(drop=True)

tabla.to_csv("resultados/resultados_modelos.csv", index=False)
tabla
```

Esta tabla **es** la "lista de resultados de todos los modelos" que pide la guía. Incluir una figura de barras: eje X = C, eje Y = RMSE, una barra por kernel.

7.9 Evaluación y figuras

- Tabla de métricas del mejor modelo.
- Scatter **predicho vs real** con la recta `y = x`.
- Histograma de **residuos** (`y_test - pred`).
- Barras comparativas de RMSE por kernel y por C.

Todas las figuras se guardan en `resultados/figuras/` con `dpi=150` y nombre descriptivo, por ejemplo `comparacion_kernels.png`.

7.10 Función de predicción para un sample nuevo

```
COLUMNAS = X_train.columns.tolist()

def predecir_venta(nuevo_sample: dict) -> dict:
    """Predice la cantidad vendida para un día y producto nuevos."""
    fila = pd.DataFrame([nuevo_sample])
    fila = fila.drop(columns=["fecha_venta"], errors="ignore")
    fila = pd.get_dummies(fila, dtype=float).reindex(columns=COLUMNAS, fill_value=0.0)

    prediccion = float(mejor_modelo.predict(fila)[0])
    return {
        "venta_estimada": round(prediccion, 1),
        "unidad": "unidades/día",
    }
```

Caso 1 — demanda alta (fin de semana, feriado, lag alto):

```
caso_alto = {
    "mes": 2, "dia_del_mes": 12, "dia_semana": 6,
    "es_fin_de_semana": 1, "es_feriado": 1, "semana_del_periodo": 7,
    "ventas_dia_anterior": 220, "ventas_hace_7_dias": 205,
    "promedio_movil_7d": 210, "stock_en_momento": 700,
    "dias_para_expirar": 150,
    "categoria": "Bebidas", "product_id": "P0008",
}
print(predecir_venta(caso_alto))
```

Caso 2 — demanda baja (entre semana, sin feriado, lag bajo):

```
caso_bajo = {
    "mes": 2, "dia_del_mes": 15, "dia_semana": 2,
    "es_fin_de_semana": 0, "es_feriado": 0, "semana_del_periodo": 7,
    "ventas_dia_anterior": 75, "ventas_hace_7_dias": 80,
    "promedio_movil_7d": 78, "stock_en_momento": 300,
    "dias_para_expirar": 120,
    "categoria": "Alimentos", "product_id": "P0004",
}
print(predecir_venta(caso_bajo))
```

Además de la predicción, mostrar el **valor real** de un registro parecido del dataset para comparar. Documentar que los dos samples pertenecen a productos y contextos distintos.

7.11 Explicabilidad

(a) Importancia global por permutación

```
from sklearn.inspection import permutation_importance

perm = permutation_importance(
    mejor_modelo, X_test, y_test,
    n_repeats=20, random_state=42,
    scoring="neg_root_mean_squared_error",
)
# Graficar las 10 variables con mayor importancia media.
```

(b) SHAP global y local

```
import shap

def predecir_array(x):
    return mejor_modelo.predict(pd.DataFrame(x, columns=COLUMNAS))

fondo = shap.sample(X_train, 100, random_state=42)
explainer = shap.KernelExplainer(predecir_array, fondo)

shap_values = explainer.shap_values(X_test.iloc[:2], nsamples=100)

shap.summary_plot(shap_values, X_test.iloc[:2], feature_names=COLUMNAS, show=False)
# Guardar figura global.

for i in range(2):
    shap.plots.waterfall(
        shap.Explanation(
            values=shap_values[i],
            base_values=explainer.expected_value,
            data=X_test.iloc[i],
            feature_names=COLUMNAS,
        ),
        show=False,
    )
    # Guardar waterfall del caso i.
```

> Usar 100 muestras de fondo y solo 2 instancias explicadas para que el cálculo sea rápido. SHAP devuelve valores en unidades de ventas porque el modelo ya desescala el target.

(c) PDP / ICE

```
from sklearn.inspection import PartialDependenceDisplay

PartialDependenceDisplay.from_estimator(
    mejor_modelo, X_train,
    ["promedio_movil_7d", "ventas_dia_anterior"],
    kind="both",
)
```

Cada figura debe acompañarse de una interpretación escrita: qué variable empuja la predicción hacia arriba o hacia abajo y por qué tiene sentido en el negocio.

7.12 Cierre: HTML y MLflow UI

```
jupyter nbconvert --to html svr_ventas.ipynb
mlflow ui --backend-store-uri ./mlruns
# abrir http://127.0.0.1:5000
```

Antes de exportar: reiniciar kernel y ejecutar todo (`Restart & Run All`) para que el HTML no tenga errores ni salidas viejas.

---

8. Convenciones de trabajo

8.1 Nombres

Elemento	Convención	Ejemplo
Notebook	snake_case	`svr_ventas.ipynb`
Experimento MLflow	snake_case	`svr_ventas`
Run MLflow	`kernel_Cvalor`	`rbf_C10`
Figura	descriptiva	`comparacion_kernels.png`
Rama	`feat/...`	`feat/modelos`
Commit	verbo + objeto	`agrega barrido de kernels con MLflow`

8.2 Calidad del notebook

- Una celda Markdown antes de cada bloque de código.
- Sin código muerto ni celdas duplicadas.
- Sin `print` de depuración en la versión final.
- Semilla fija en todo lo aleatorio.
- El notebook debe correr de inicio a fin en menos de 5 minutos.
- Gráficos con título, ejes rotulados y unidades.

8.3 Git

```
git checkout -b feat/modelos
git add svr_ventas.ipynb
git commit -m "agrega barrido de kernels y C con MLflow"
git pull --rebase origin main
git push origin feat/modelos
```

- Nunca commitear `.venv/`, `mlruns/` completo con artefactos pesados ni `__pycache__/`. Agregar `.gitignore` con `.venv/`, `mlruns/`, `__pycache__/`, `.ipynb_checkpoints/`.
- Los artefactos de MLflow se conservan localmente para la demostración; si el docente pide el repositorio, se documenta cómo regenerarlos.

8.4 Comunicación

- Un canal único del equipo (WhatsApp/Discord) para dudas y avisos.
- Reunión de 10 minutos al inicio de cada sesión: qué se hizo, qué sigue, bloqueos.
- Si alguien no puede asistir, avisa con 24 h y deja su parte documentada.
- Las decisiones técnicas se registran en la sección 3 de este documento, no en el chat.

9. Criterios de aceptación (checklist de rúbrica)

Marcar antes de la entrega:

- [ ] **3.1** Se entrenaron los 4 kernels y al menos 4 valores de C.
- [ ] **3.1** Todos los runs aparecen en `mlflow ui` con parámetros y métricas.
- [ ] **3.1** Existe una tabla comparativa (`resultados_modelos.csv`) y figuras que resaltan el mejor modelo.
- [ ] **3.1** `GridSearchCV` documentado con los mejores hiperparámetros.
- [ ] **3.1** Métricas reportadas: RMSE, MAE, R^2 y MAPE.
- [ ] **3.2** Permutation importance con gráfico e interpretación.
- [ ] **3.2** SHAP global (summary) y local (waterfall) para los dos casos.
- [ ] **3.2** PDP/ICE de al menos dos variables.
- [ ] **3.2** Cada figura tiene una explicación en Markdown.
- [ ] **3.3** `predecir_venta()` definida y ejecutada con dos casos distintos.
- [ ] **3.3** Se muestra la predicción y se compara con un valor real.
- [ ] **3.4** Conclusiones con métricas concretas y comparación con la práctica 001.
- [ ] **3.4** Referencias en APA 7, citadas en el texto.
- [ ] Notebook ejecutado de inicio a fin sin errores.
- [ ] `svr_ventas.html` generado y revisado.
- [ ] Repositorio ordenado y `.gitignore` correcto.
- [ ] Todos los integrantes pueden explicar cualquier sección.

10. Riesgos y mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Conflictos al editar el notebook	Alta	Medio	Ramas por persona, secciones asignadas, integrador único
SHAP lento o consume mucha memoria	Media	Medio	100 muestras de fondo, 2 instancias, `nsamples=100`
Kernel `sigmoid` inestable	Alta	Bajo	Documentarlo como resultado esperado; limitar C
Métricas distintas entre integrantes	Media	Medio	Semilla fija, split congelado, mismos datos
Fuga de información en features	Media	Alto	Revisión cruzada P1–P2 antes de modelar
Split temporal con solo ~100 filas de test	Alta	Medio	Reportar también CV 5-fold; no sobreinterpretar decimales
MLflow no registra artefactos	Baja	Medio	Verificar en S2 con un run de prueba
Pérdida de una sesión	Media	Medio	Sesión buffer S6
Un integrante se atrasa	Media	Alto	Tareas independientes y revisión por pares

11. Plan de sustentación

Duración total: 12–15 minutos + preguntas. Un bloque por integrante.

Bloque	Minutos	Contenido	Responsable
1. Contexto	3	Dataset, variable objetivo, por qué SVR, split y control de fuga	P1
2. Modelo y optimización	4	Pipeline, kernels, C, MLflow, GridSearch, tabla de resultados	P2
3. Evaluación y predicción	3	RMSE/MAE/R^2, figuras, `predecir_venta()` y los dos casos	P3
4. Explicabilidad y conclusiones	4	SHAP, PDP, hallazgos, limitaciones, referencias	P4

Preguntas probables y respuestas cortas

- **¿Por qué SVR y no clasificación?** Porque el target `cantidad_vendida` es continuo y la guía permite regresión; inventar clases perdería información.
- **¿Por qué escalar el target?** Porque `epsilon` y `C` dependen de la escala; sin escalar, el modelo sobreajusta.
- **¿Por qué split temporal?** Para evitar que días consecutivos queden en train y test, lo que inflaría las métricas.
- **¿Por qué `sigmoid` falla?** No es un kernel adecuado para este tipo de datos tabulares; produce predicciones extremas.
- **¿Qué significa un SHAP positivo?** Que esa variable empuja la predicción por encima del valor base del modelo.
- **¿El modelo sirve para producción?** No tal cual: hay 60 días y 10 productos; se necesita más histórico y validación temporal más larga.

---

12. Referencias (APA 7)

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.

Molnar, C. (2022). *Interpretable machine learning: A guide for making black box models explainable* (2.ª ed.). https://christophm.github.io/interpretable-ml-book/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Raschka, S. (2017). *Python machine learning* (2.ª ed.). Packt Publishing.

Smola, A. J., & Schölkopf, B. (2004). A tutorial on support vector regression. *Statistics and Computing, 14*(3), 199–222. https://doi.org/10.1023/B:STCO.0000035301.49549.88

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nystrom, T., Ogilvie, P., Parkhe, M., Plawinski, M., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin, 41*(4), 39–45.

13. Anexos

A. Plantilla de tabla de resultados

Run	Kernel	C	Epsilon	Gamma	RMSE	MAE	R^2	MAPE
linear_C0.1	linear	0.1	0.1	—				
linear_C1	linear	1	0.1	—				
rbf_C10	rbf	10	0.1	scale				
poly_C10	poly	10	0.1	scale				
sigmoid_C10	sigmoid	10	0.1	scale				

B. Checklist de revisión final

1. Reiniciar kernel y ejecutar todo el notebook.
2. Verificar que no haya warnings graves ni celdas con error.
3. Revisar que las 16+ runs estén en MLflow.
4. Abrir `resultados_modelos.csv` y confirmar el orden por RMSE.
5. Revisar que cada figura esté citada e interpretada en Markdown.
6. Confirmar que los dos casos de predicción usan productos y contextos distintos.
7. Revisar ortografía y formato APA.
8. Exportar HTML y abrirlo en el navegador.
9. Ensayar la sustentación completa una vez.

C. Comandos útiles

```
# Activar entorno
source .venv/bin/activate

# Ejecutar el notebook desde la terminal
jupyter nbconvert --to notebook --execute --inplace svr_ventas.ipynb

# Exportar a HTML
jupyter nbconvert --to html svr_ventas.ipynb

# Ver experimentos
mlflow ui --backend-store-uri ./mlruns

# Revisar estado del repositorio
git status
git log --oneline --graph --decorate
```

D. Glosario mínimo

- **SVR:** Support Vector Regression, SVM adaptada a variables continuas.
- **Kernel:** función que mapea los datos a un espacio de mayor dimensión.
- **C:** penalización por error; valores altos ajustan más y pueden sobreajustar.
- **Epsilon:** ancho de la banda de tolerancia del SVR.
- **Gamma:** alcance de influencia de cada punto en el kernel RBF.
- **SHAP:** valores de Shapley que reparten la predicción entre las variables.
- **PDP:** Partial Dependence Plot, efecto promedio de una variable sobre la predicción.
- **ICE:** Individual Conditional Expectation, efecto por instancia.
- **Fuga de información:** usar variables que contienen información del target o del futuro.

Nota: este documento es el plan de desarrollo. La implementación del notebook se realiza en el agente de construcción; aquí solo se definen decisiones, responsables, cronograma y criterios de aceptación.